

AI/ML Assessment (Version B) — Q&A with Explanations

Condensed answer key: question, correct answer, and why — Section A (MCQ) and Section B (Theory & Coding, summarised)

Section A — Multiple Choice

Q1.

A DataFrame orders has columns customer_id and order_value. Which expression returns the average order value per customer, keeping only customers who placed more than five orders?

Answer: C) `orders.groupby('customer_id').filter(lambda g: len(g) > 5).groupby('customer_id')['order_value'].mean()`

This filters groups (customers) with more than 5 rows first, then computes the mean. B wrongly filters individual order values >5, not order count. D checks if the mean itself is >5, not the count.

Q2.

LEFT JOIN combined with WHERE o.status = 'shipped' — which statement is correct?

Answer: B) The WHERE clause removes rows where o.status is NULL, so it behaves like an INNER JOIN

WHERE runs after the join. NULL never equals 'shipped', so customers with no shipped order (including no orders at all) get filtered out — the LEFT JOIN effectively behaves like an INNER JOIN here.

Q3.

A client's request times out and it's unclear if the server processed it. Which statement about retrying is correct?

Answer: A) PUT is idempotent; POST is not and may create a duplicate record

Repeating PUT leaves the resource in the same end state, so it's safe to retry. Repeating POST risks creating a duplicate (e.g. a second order).

Q4.

Which of the following is an example of data leakage?

Answer: D) Including days_since_account_frozen, when accounts are only frozen after a customer has already defaulted

This feature only exists because the outcome already happened — it encodes future/target information directly into the input.

Q5.

Validating a model on 3 years of loan data that will score future applications — best validation strategy?

Answer: B) TimeSeriesSplit

Trains on earlier data and validates on strictly later data, mirroring real deployment. Random splits let future information leak into training.

Q6.

A spam filter wrongly sending legitimate email to junk is far more costly than letting spam through. Which metric to prioritise?

Answer: A) Precision on the spam class

Precision penalises false positives (legit email wrongly flagged as spam), which is exactly the costly error described here.

Q7.

Training accuracy 61%, validation accuracy 60%, both flat for many epochs. Which action helps most?

Answer: C) Increasing model capacity, or training longer with a better learning rate

Low-and-close train/val accuracy is a classic underfitting signature (high bias), not overfitting. More regularisation (A) would make underfitting worse.

Q8.

Elongated, crescent-shaped clusters with scattered outliers between them — which statement is true?

Answer: D) DBSCAN is better suited, because it finds arbitrarily shaped clusters and labels low-density points as noise

DBSCAN groups by density and handles non-convex shapes well. K-Means assumes roughly spherical clusters and performs poorly here.

Q9.

Tensor (32, 200, 6) through `nn.LSTM(input_size=6, hidden_size=64, num_layers=2, batch_first=True)` — shape of the output sequence?

Answer: B) (32, 200, 64)

`batch_first=True` keeps the output as (batch, seq_len, hidden_size) — one 64-dim output per timestep. (2, 32, 64) would instead be the final hidden state `h_n`, not the output sequence.

Q10.

Binary classifier, final layer outputs a single raw value per sample. Which is correct?

Answer: A) Pass the raw value to `nn.BCEWithLogitsLoss`, which applies sigmoid internally and is numerically more stable

`BCEWithLogitsLoss` uses a numerically stable computation internally, avoiding precision issues from applying sigmoid separately first.

Q11.

Why does doubling input sequence length more than double the compute cost of self-attention?

Answer: C) Every token attends to every other token, so the attention matrix grows with the square of sequence length

Attention computation scales as n^2 . Doubling the sequence length n gives roughly $4\times$ the cost, not $2\times$.

Q12.

RAG corpus indexed 6 months ago; query-side upgraded to a newer embedding model but corpus not re-indexed. What happens?

Answer: D) Retrieval quality collapses, because query and stored vectors no longer occupy the same space; the corpus must be re-embedded

Different embedding models produce incompatible vector spaces. Old document vectors and new query vectors no longer share a common geometry for similarity search.

Q13.

Which scenario is the strongest case for fine-tuning rather than RAG?

Answer: B) Every response must follow a rigid internal report structure and house tone, whatever the subject

This is a behaviour/style requirement, not a need for fresh factual retrieval — exactly what fine-tuning is suited for. The other options all need up-to-date or citable facts, which is RAG's strength.

Q14.

An agent has a tool available but never calls it, answering from its own knowledge instead. Most likely cause?

Answer: A) The tool's description does not make clear when the tool should be used

Models decide whether to call a tool mainly from its name/description matching the current need. An unclear description means the model won't recognise it's relevant.

Q15.

An HTTP Request node in n8n intermittently fails when the upstream API is briefly unavailable. Appropriate handling?

Answer: C) Enable Retry On Fail with a retry count and wait interval, and route persistent failures to an error branch or error workflow

This is n8n's purpose-built mechanism for exactly this situation — recovering from transient failures while surfacing genuine, persistent ones.

Section B — Theory & Coding (Condensed)

Q16.

Q16(a). How should applications.csv be split for training, and why?

Answer: Split chronologically — train on earlier applications, test on the most recent portion.

The model scores future applications, so a random split would let later rows leak into training, letting the model implicitly learn patterns it wouldn't have access to yet in production. A Pipeline with ColumnTransformer then handles imputation, scaling, and encoding in one object so preprocessing is applied consistently to train and test.

Q17.

Q16(b). Two problems with `train_test_split(random_state=42)` reporting 94% accuracy?

Answer: (1) Random split leaks future data into training. (2) Accuracy is misleading on an imbalanced (12% default) dataset.

A model predicting 'No' for everyone would already score ~88% accuracy, so 94% reflects little real predictive skill. Precision/recall/F1/AUC on the minority class are needed instead of raw accuracy.

Q18.

Q16(c). Given defaults cost █80k vs █6k for wrongly rejecting a good applicant, which metric to optimise?

Answer: A recall-weighted metric such as F2-score (beta=2), tuned via GridSearchCV(scoring=f2_scorer).

The ~13:1 cost asymmetry means missing a defaulter (false negative) is far more expensive than wrongly rejecting a good applicant (false positive), so recall should be weighted more heavily than precision.

Q19.

Q17(a). Which LSTM output feeds the final dense layer, and why?

Answer: The last timestep's output, `out[:, -1, :]`, feeding a `Linear(64, 4)` layer.

The LSTM processes the sequence in order, so by the final timestep its hidden state has already incorporated information from all 200 earlier timesteps — a natural summary of the whole sequence for classification.

Q20.

Q17(b). What do `zero_grad()`, `backward()`, and `step()` do; and `eval()` vs `no_grad()`?

Answer: `zero_grad()` clears old gradients, `backward()` computes new ones via backprop, `step()` updates weights. `eval()` changes layer behaviour (dropout/batchnorm); `no_grad()` disables gradient tracking for efficiency.

PyTorch accumulates gradients by default, so `zero_grad()` prevents them stacking across batches. `eval()` alone doesn't stop gradient tracking, and `no_grad()` alone doesn't fix dropout/batchnorm behaviour — both are needed together during validation.

Q21.

Q17(c). 71% val accuracy, one class is 71% of the data, model predicts that class every time. What happened, and how to fix it?

Answer: Majority-class collapse. Fix with class-weighted loss (or oversampling) and monitor macro-F1/confusion matrix instead of raw accuracy.

Loss still dropped during training because average cross-entropy genuinely improves as the model gets more confident on the dominant class, even while learning nothing about the other three fault types.

Q22.

Q18(a). How do you architect a multi-tenant RAG system with follow-up questions?

Answer: Ingest: chunk, embed, store with client_id metadata. Query: rewrite follow-ups into standalone questions, embed, retrieve with a hard client_id filter, then generate.

The client_id filter must be a hard pre-filter at the vector-database query, derived server-side from the authenticated session (never client-supplied), so tenants can never see each other's documents. A query-rewriting step resolves references like 'what about for contractors?' into a self-contained question before retrieval.

Q23.

Q18(c). Three retrieval-stage failure causes, and when is fine-tuning better than RAG?

Answer: Unrewritten follow-ups, chunks splitting an answer across boundaries, and stale embeddings after a model upgrade. Fine-tuning wins for consistent structure/tone, not fresh facts.

Each cause maps to a concrete fix: query rewriting, structure-aware/larger overlapping chunks or parent-document retrieval, and re-indexing whenever the embedding model changes.

Q24.

Q19(a)-(b). Key FastAPI + Docker deployment decisions?

Answer: Load the model once at startup (not per request); validate batch size with Pydantic (max_items=500); use a slim base image; install requirements.txt before copying app code; run as a non-root user.

Installing dependencies before app code lets Docker reuse cached layers on rebuilds when only code changes. Non-root limits the impact if the container is ever compromised.

Q25.

Q19(c). More rejections after v3 deploy, no errors thrown — how to investigate, and skew vs drift?

Answer: Shadow-compare v3 vs v2 predictions on the same recent batch; check for silent feature/encoding changes; roll back via a versioned model registry.

Training-serving skew is a pipeline inconsistency between how features are computed at train time vs serve time. Data drift is the real-world input distribution genuinely changing over time, even though the pipeline itself stayed consistent.

Q26.

Q20(a). What's the n8n node sequence for the invoice-processing workflow?

Answer: Email Trigger → AI extraction node → HTTP Request (PO lookup) → IF/Switch (±2% match) → match: post to accounting + send acknowledgement / mismatch: Slack to finance approver, no acknowledgement.

Data flows from raw email+PDF → structured invoice JSON → merged with the PO lookup response → a boolean match decision that branches the workflow.

Q27.

Q20(b). How does the AI Agent decide to call the purchase-order lookup tool?

Answer: It emits a tool call with supplier_name (and po_number if known) when it judges the task needs PO data; reads the returned JSON; stops calling tools once it can produce a final match/mismatch decision.

The tool schema's description is what signals to the model when the tool is relevant — an unclear description (as in Q14) is the most common reason an agent skips a tool it should use.

Q28.

Q20(c). How to handle malformed LLM JSON, duplicate invoices, and scanned images with no text layer?

Answer: Malformed JSON: retry/self-repair or route to Slack. Duplicates: idempotency check on invoice_number + supplier before posting. Scanned images: OCR or a vision-capable model before extraction.

Each of these is a production edge case a demo workflow would silently break on — naming the concrete fallback for each is what separates a 'happy path only' answer from a production-ready one.

For full runnable code on every Section B part, see the detailed answer key PDF shared earlier.